

Informe Técnico: Escáner Léxico para C

Shania Maía Russo De Ávila (2021114030)
Juan Francisco González Pacheco (2021114063)
Arlinson Adalberto Barrios Aguilar (2023114035)
Isabel Duran Alvear (2022114033)
Iván Camilo Alarcón Moreno (2021214036)

3 de mayo de 2025

Presentado a: Ing. Docente Esmeide Leal Narvaez
Universidad Del Magdalena
Facultad De Ingeniería
Programa de Ingeniería de Sistemas
Compiladores - Segundo Seguimiento

1 Introducción

El escáner léxico transforma código fuente en C en tokens, como palabras reservadas, identificadores, números y operadores, para el análisis sintáctico de un compilador. Utiliza Autómatas Finitos Deterministas (AFD) para validar identificadores y números, incluyendo enteros, números de punto flotante y notación científica.

2 Tipo de Autómata Diseñado

El proyecto emplea Autómatas Finitos Deterministas (AFD) en tres clases:

- `NotacionCientifica.java`: Valida números enteros, de punto flotante y en notación científica con un AFD de 10 estados.
- `VariablesC.java`: Verifica identificadores con un AFD de 3 estados.
- `EscanerC.java`: Usa transiciones condicionales para simular un AFD implícito.

Justificación:

- **Determinismo**: Cada transición es única, sin ambigüedad.
- **Eficiencia**: Evita retroceso, a diferencia de un AFND.
- **Simplicidad**: Las tablas de transiciones son fáciles de implementar.

3 Diseño de los Autómatas

3.1 Autómata para Números (`NotacionCientifica.java`)

Valida números como 123, 3.14 y 1.23e-4, rechazando entradas inválidas como 12..3.

Estados:

- 0: Inicial (espera dígito, signo o punto).
- 1–8: Intermedios (parte entera, decimal, exponente).
- 9: Error.
- Aceptación: 2 (dígito), 3 (dígito punto), 4 (dígito punto dígito), 6 (dígito punto exponencial dígito – Notación científica).

Transiciones:

- Matriz `tablaTransiciones` con columnas para dígitos, punto, exponente (E/e), signo (+/-) y otros.

Validación:

- Limita a un punto decimal.
- Acepta 3.14, 1.23e-4; rechaza 12..3.

3.2 Autómata para Identificadores (`VariablesC.java`)

Valida identificadores como `variable_1` y rechaza `1variable`.

Estados:

- 0: Inicial (letra o `_`).
- 1: Aceptación.
- 2: Error.

Transiciones:

- Columnas: Letras, dígitos, `_`.

Validación:

- Acepta `myVar`; rechaza `var@name`.

4 Diseño de la Función de Escaneo (EscanerC.java)

El escáner procesa el código fuente carácter por carácter, clasificando tokens mediante lógica condicional.

4.1 Lectura del Archivo

- Usa `BufferedReader` para leer líneas.
- Omite espacios con `Character.isWhitespace`.

4.2 Reconocimiento de Tokens

- **Cadenas:** Recolecta texto entre comillas dobles (`STRING`).
- **Palabras Reservadas e Identificadores:** Agrupa letras y `_`, verificando con `getPalabraReservada` o `VariablesC.esIdentificadorValido`.
- **Números:** Recolecta dígitos, puntos y exponentes, validando con `NotacionCientifica` (`NUMBER`).
- **Operadores:** Prioriza operadores dobles (`==`) sobre simples (`+`) con `getSimboloEspeci`

4.3 Manejo de Errores

- Omite caracteres desconocidos (`@`).
- Maneja cadenas incompletas sin validar.
- Filtra tokens inválidos mediante autómatas.

4.4 Ejemplo de Tokens

- **Ejemplo 1:** `int c = 3.14;`
 - `INT "int", ID "c", ASSIGN "=", NUMBER "3.14", SEMI ";"`
- **Ejemplo 2:** `if (x <= 10) printf("Hola");`
 - `IF "if", LPAR "(", ID "x", LTEQ "<=", NUMBER "10", RPAR ")", WRITE "printf", STRING "Hola", SEMI ";"`

5 Integración y Ejecución (Main.java)

Main.java ejecuta el escáner, llamando a `escanearArchivo` con un archivo en C, mostrando los tokens en consola.

6 Pruebas y Validación

Usamos el código de test1.c, que es:

1	<code>#include <iostream></code>
2	<code>using namespace std;</code>
3	
4	<code>int main() {</code>
5	<code> int a;</code>
6	<code> int b;</code>
7	<code> a = b + 1;</code>
8	<code> int c = 3.14;</code>
9	<code> cout<<"hello";</code>
10	<code> return 0;</code>
11	<code>}</code>

Lo cual genera la siguiente salida por consola:

Token: ID "include"
Token: LT "<"
Token: ID "iostream"
Token: GT ">"

Token: ID "using"
Token: ID "namespace"
Token: ID "std"
Token: SEMI ";"
Token: INT "int"
Token: MAIN "main"
Token: LPAR "("
Token: RPAR ")"
Token: LBRACE "{"
Token: INT "int"
Token: ID "a"
Token: SEMI ";"
Token: INT "int"
Token: ID "b"
Token: SEMI ";"
Token: ID "a"
Token: ASSIGN "="
Token: ID "b"
Token: PLUS "+"
Token: NUMBER "1"
Token: SEMI ";"
Token: INT "int"
Token: ID "c"
Token: ASSIGN "="
Token: NUMBER "3.14"
Token: SEMI ";"
Token: ID "cout"
Token: SHL_OP "<<"
Token: STRING "hello"

Token: SEMI ";"

Token: RETURN "return"

Token: NUMBER "0"

Token: SEMI ";"

Token: RBACE "}"

BUILD SUCCESSFUL (total time: 0 seconds)